CSE 503 – Advanced Operating Systems

Rashid Jillani
rjillani@giki.edu.pk

Course Information

Instructor: Dr. Rashid Jillani

- Faculty of Computer Science and Engineering
- Email: rjillani@giki.edu.pk
- Office: G-05, FCSE

- Office Hours:

Tuesday - Friday	**9:00 AM - 5:00 PM**

Schedule

- Wednesday : 03:50 pm – 04:10 pm CS-LH2
- Friday : 03:50 pm – 04:10 am CS-LH2

Prerequisite: Operating System

Course material: MS Teams (**y1smb3n**)

Tentative Course Outline

- Introduction (Week 1)
- Processes and Threads (Week 2-3)
- Memory Management (Week 2)
- Input/Output (Week 1-2)
- File Systems (Week 2)
- Deadlocks (Week 1-2)
- Virtualization and Cloud Computing (Week 2)
- Multiple Processor Systems (Week 1)
- Protection and Security (Week 1)
- Case studies: (Parallel)
  - Linux
  - Android

Tentative Assessment

- Assignments 05 %
- Quizzes 20 %
- Mid Term 30 %
- Final Exam 45 %
- **Total** **100 %**

Plagiarism Policy

Plagiarism in assignments may lead you towards 'F' grade in this course :(

Text Books

1. Modern Operating Systems by Andrew S. Tanenbaum, 4th Edition
2. Operating Systems Internals and Design Principles, by William Stallings, 7th Edition
3. Operating System Concepts, by Abraham Silberschatz et. al., 8th Edition

References

1. Operating Systems – Design and Implementation, by Andrew S. Tanenbaum and Albert S. Woodhull

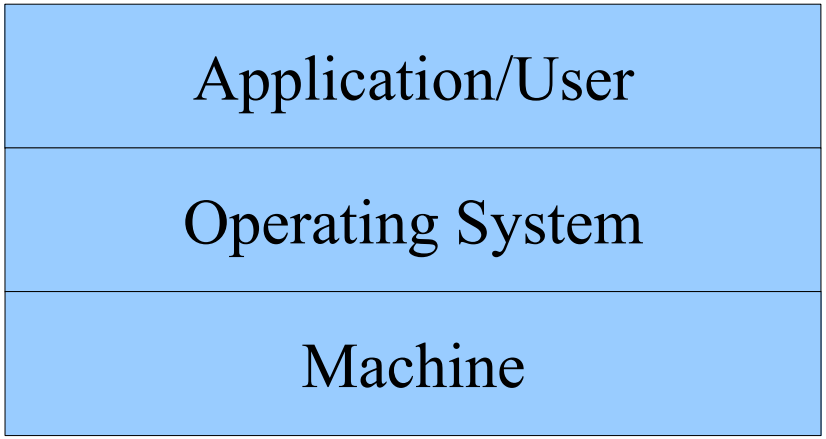
2. Distributed Systems – Principles and Paradigms, by Andrew S. Tanenbaum and Maarten Van Steen, 2nd Edition.

Chapter 1

Introduction

1.1 What is an operating system
1.2 History of operating systems
1.3 The operating system zoo
1.4 Computer hardware review
1.5 Operating system concepts
1.6 System calls
1.7 Operating system structure

Introduction



An Operating System is a software layer, whose job is to provide user programs with a better, simpler, cleaner, model of the computer and to handle managing all the ***resources***.

Introduction

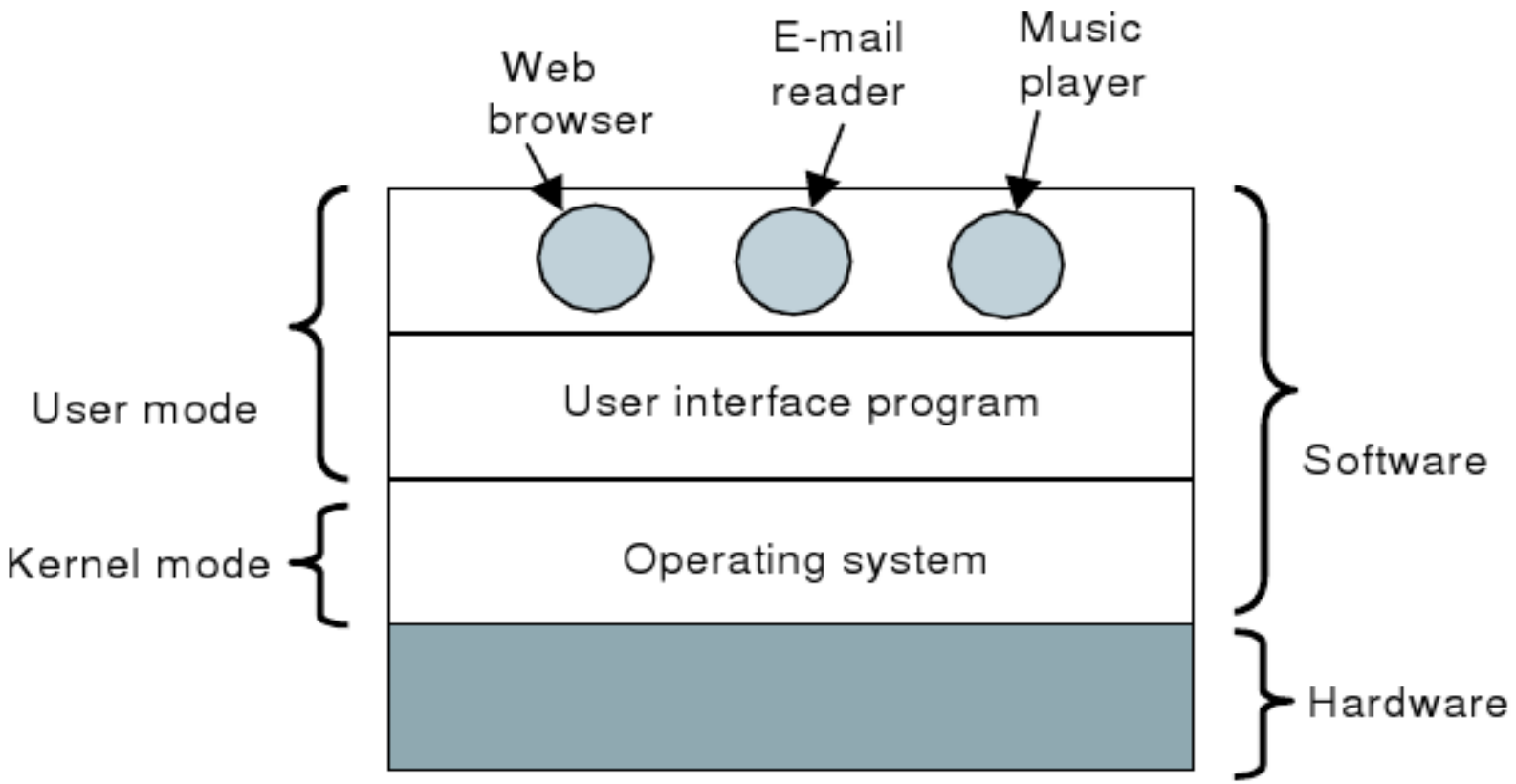


Figure 1-1. Where the operating system fits in.

Interface

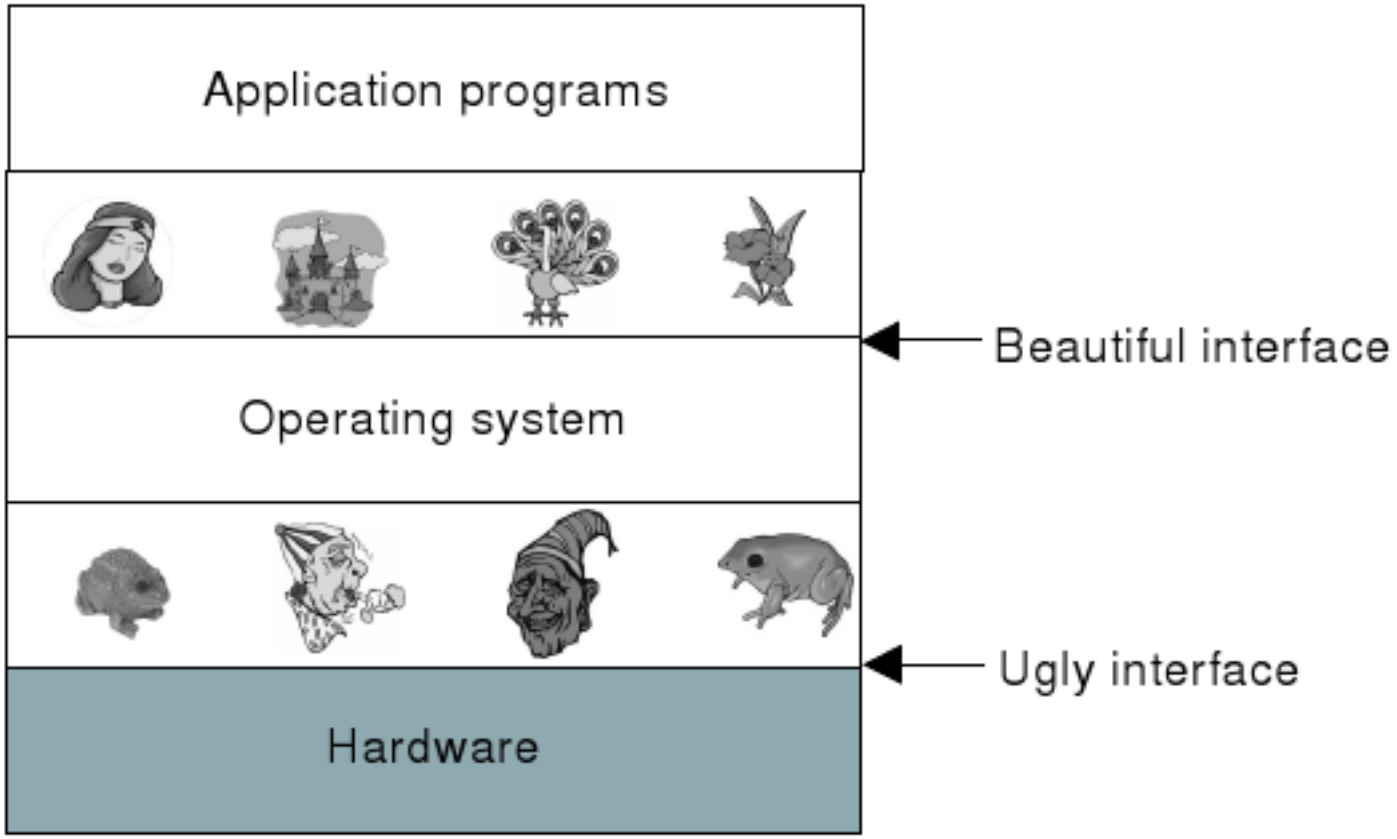
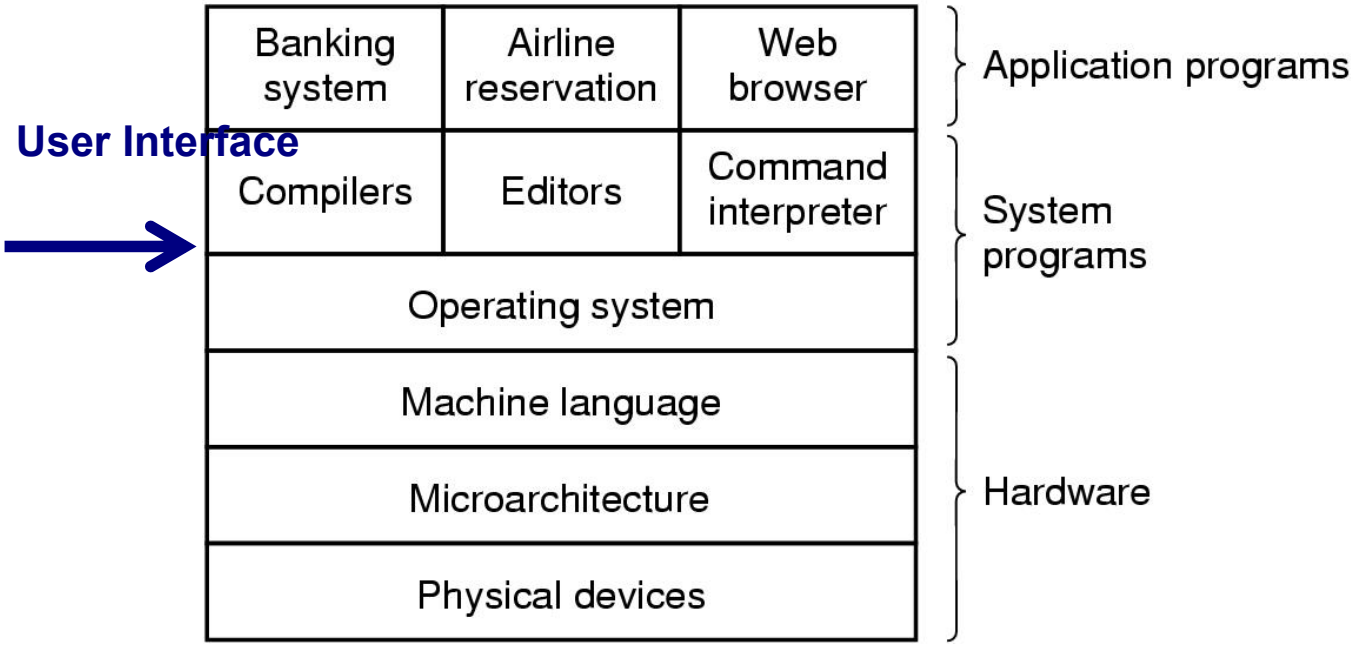


Figure 1-2. Operating systems turn ugly hardware into beautiful abstractions.

Introduction...



- A computer system consists of
  - hardware
  - system programs
  - application programs

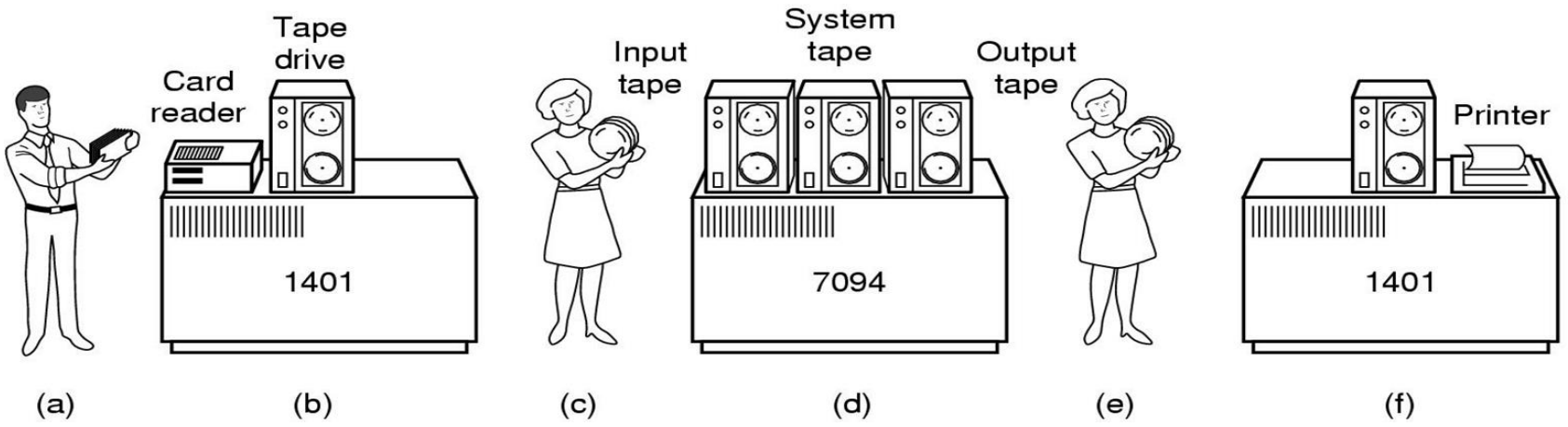
Operating System Complexity

- 50,000,000 lines of code
- A book of 100 volumes
- Each volume contains 1000 pages
- Each page contains 50 lines
- An entire book case

What is an Operating System

- It is an extended machine
  - –Hides the messy details which must be performed
  - –Presents user with a virtual machine, easier to use
- It is a resource manager
  - –Each program gets time with the resource
  - –Each program gets space on the resource

History of Operating Systems (1)



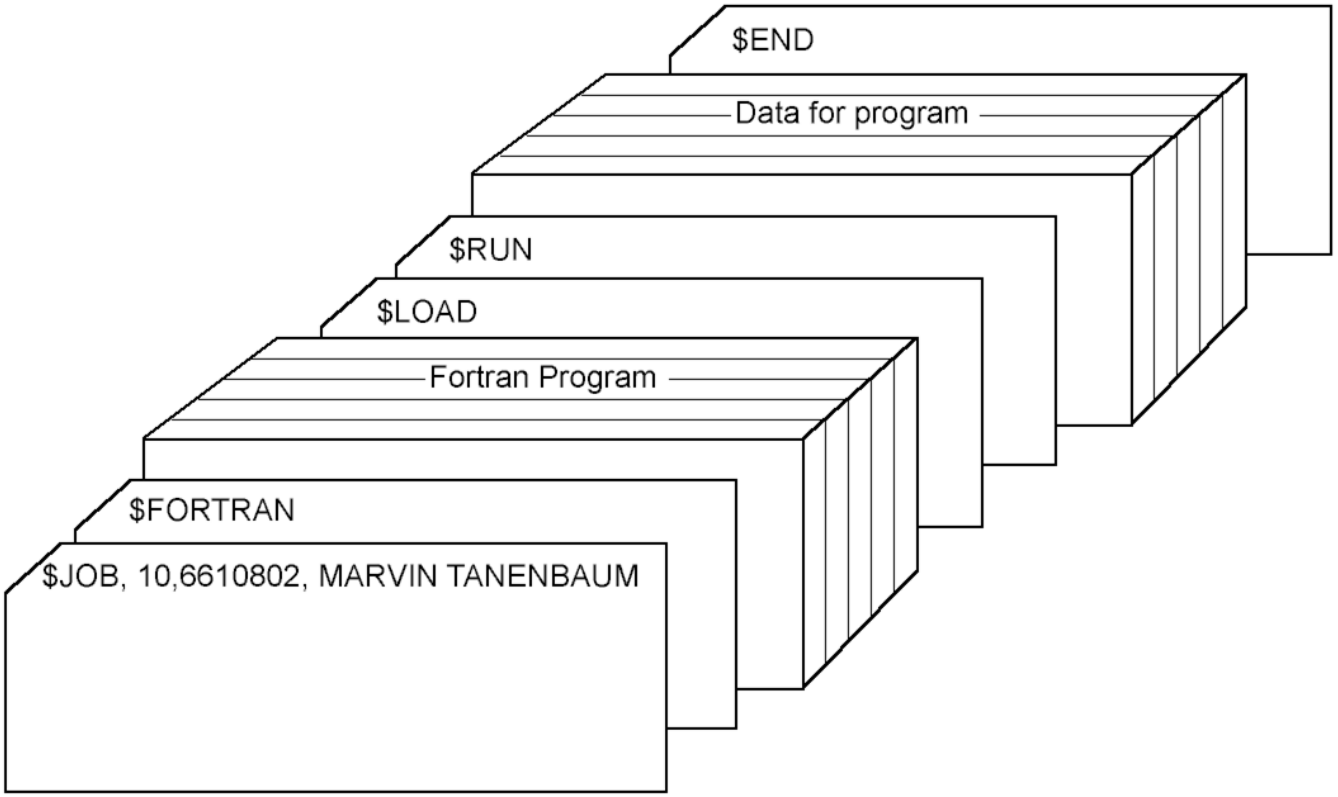
Early batch system

- –bring cards to 1401
- –read cards to tape
- –put tape on 7094 which does computing
- –put tape on 1401 which prints output

History of Operating Systems (2)

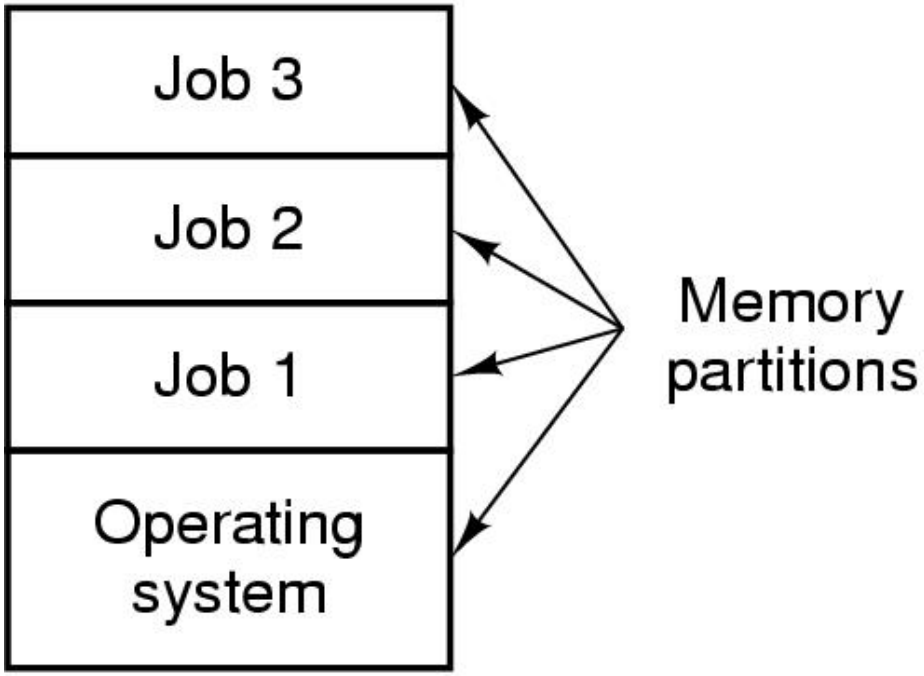
- First generation 1945 - 1955
  - –Vacuum tubes, plug boards
- Second generation 1955 - 1965
  - –Transistors, batch systems
- Third generation 1965 – 1980
  - –ICs and multiprogramming
- Fourth generation 1980 – present
  - –Personal computers
- Fifth generation 1990 – present
  - –Mobile computers

History of Operating Systems (3)



Structure of a typical FMS job – 2nd generation

History of Operating Systems (4)



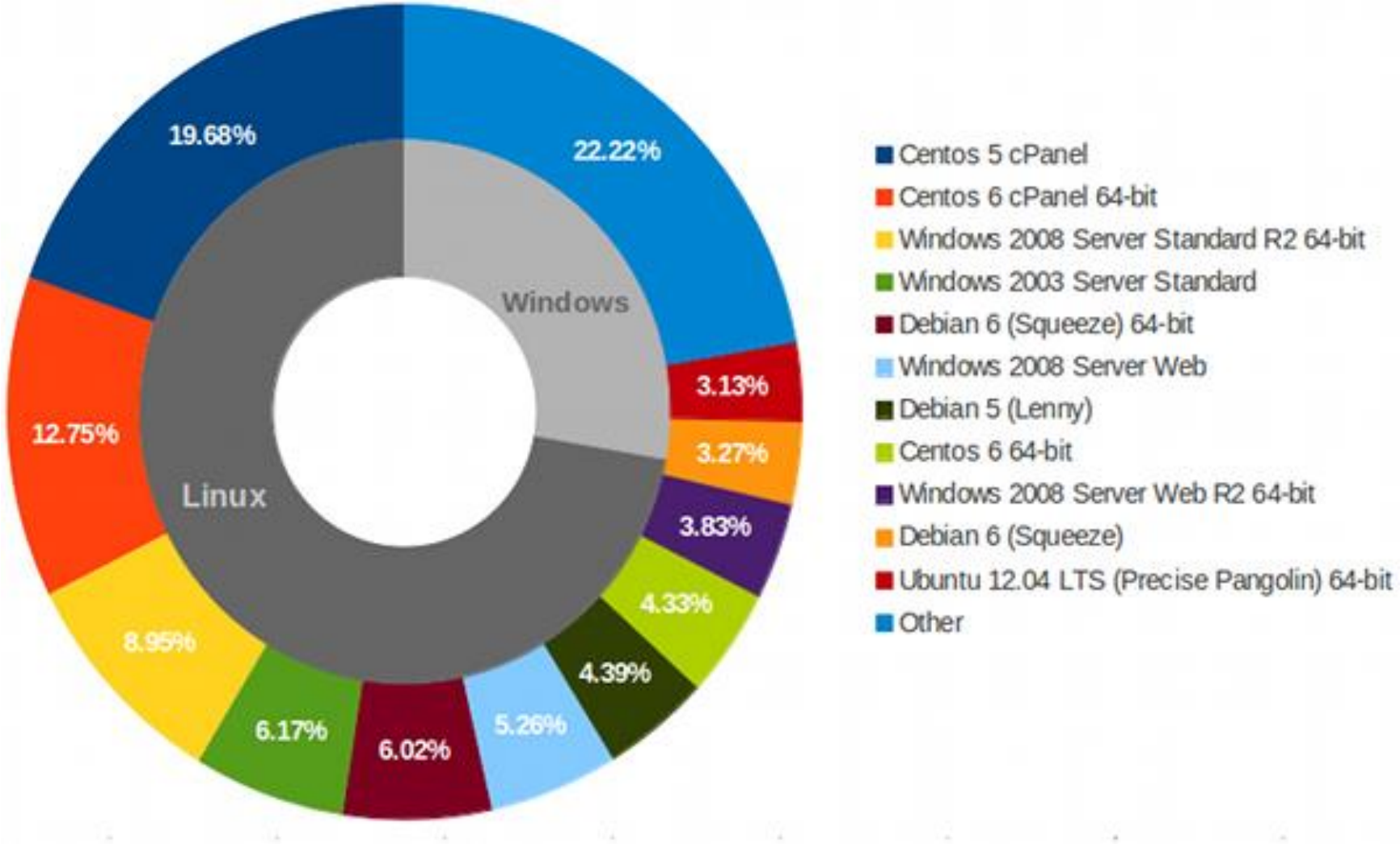
Multiprogramming system

–three jobs in memory – 3rd generation

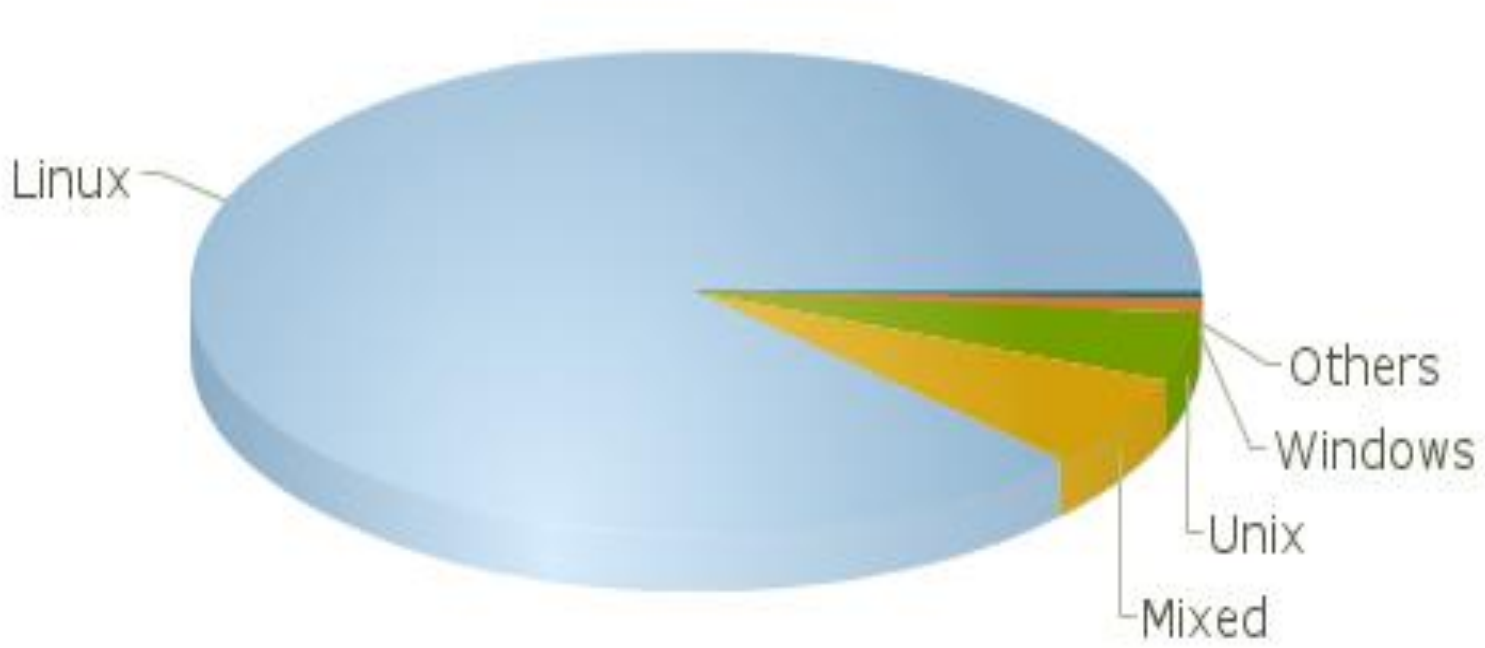
The Operating System Zoo

- Mainframe operating systems
- Server operating systems
- Sensor-Node operating systems
- Multiprocessor operating systems
- Personal computer operating systems
- Real-time operating systems
- Embedded operating systems
- Smart card operating systems

Current Top 500 OS (1)



Current Top 500 OS (2)



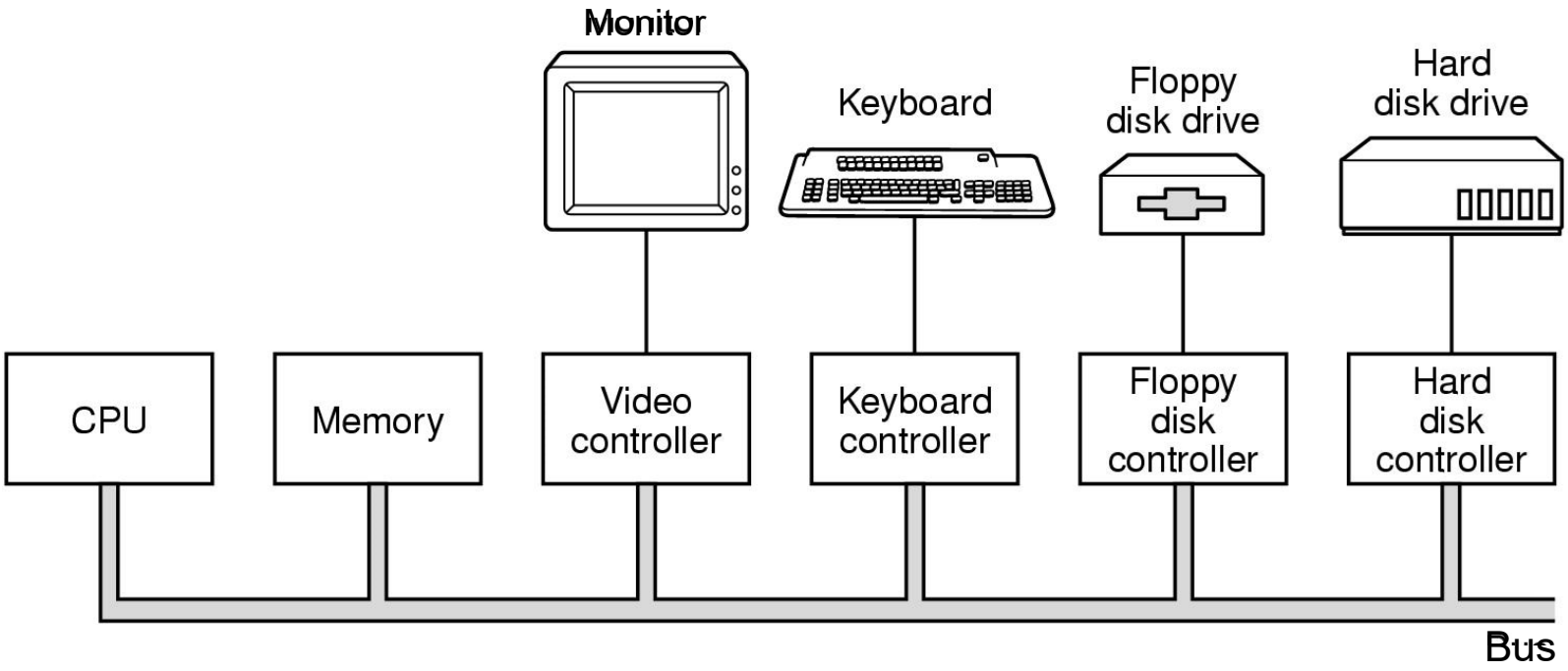
Operating System Family

Current Top 500 OS (3)

Operating system Family	Count	Share %
Linux	439	87.80 %
Windows	5	1.00 %
Unix	23	4.60 %
BSD Based	1	0.20 %
Mixed	31	6.20 %
Mac OS	1	0.20 %
Totals	**500**	**100%**

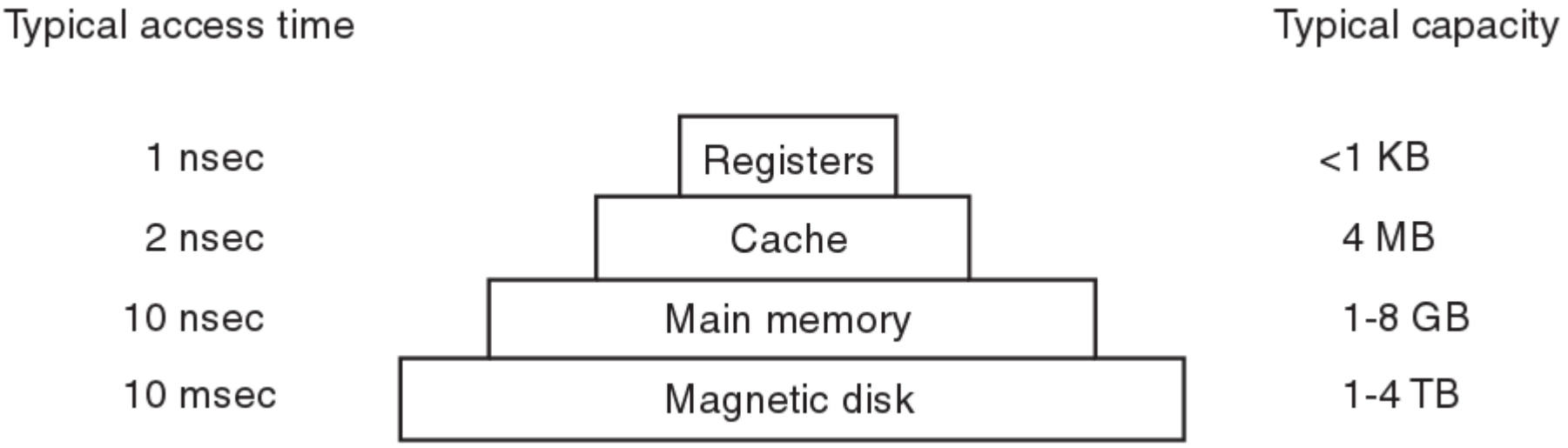
Operating System Family

Computer Hardware Review (1)



Components of a simple personal computer

Computer Hardware Review (2)



Typical memory hierarchy

–numbers shown are rough approximations

Flynn's taxonomy

- Single Instruction, Single Data stream (SISD)

  – Traditional uniprocessor machines like a PC or old mainframes.
- Single Instruction, Multiple Data streams (SIMD)

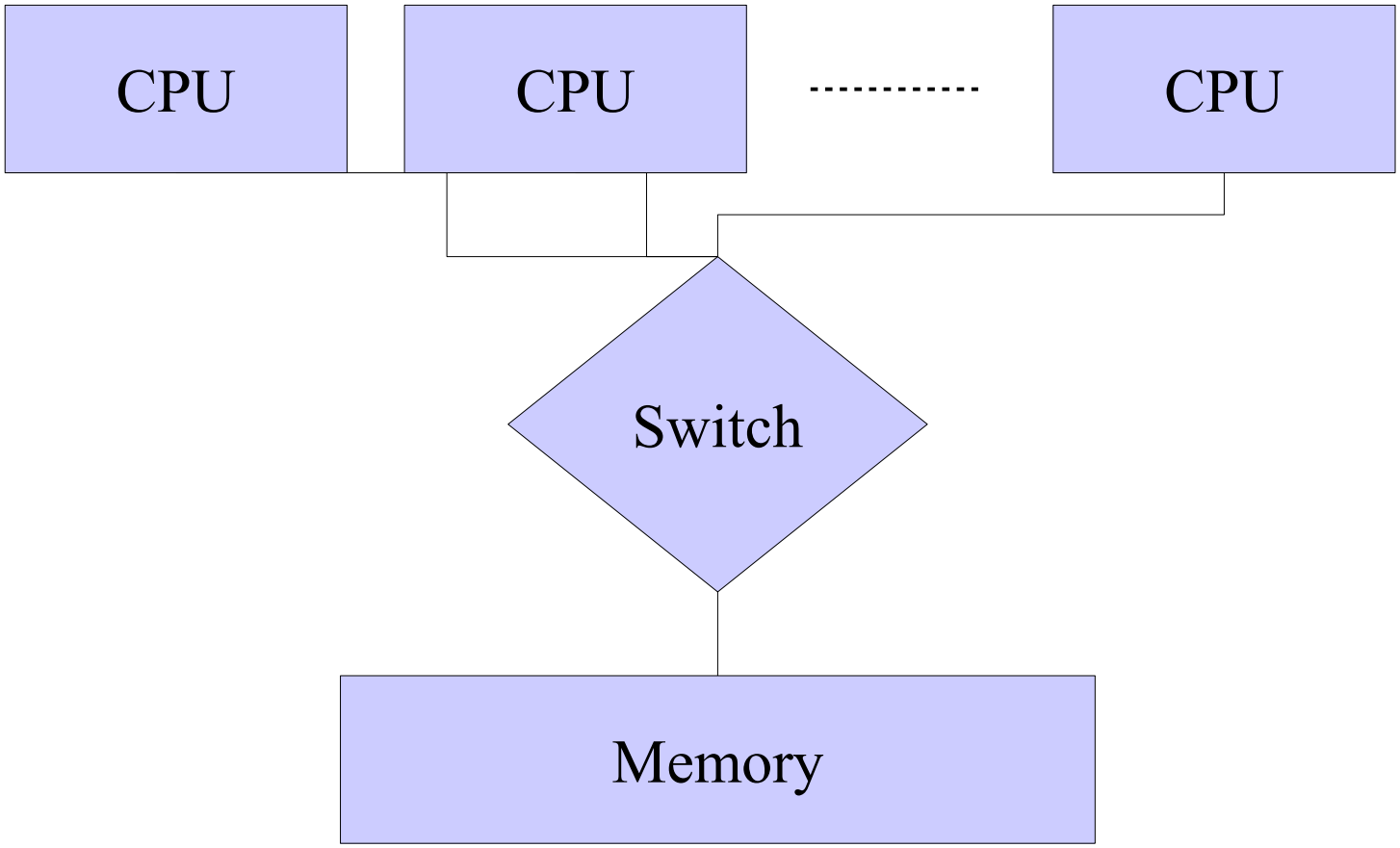
  –For example, an array processor or GPU.
- Multiple Instruction, Single Data stream (MISD)

  –No known implementation so far.
- Multiple Instruction, Multiple Data streams (MIMD)

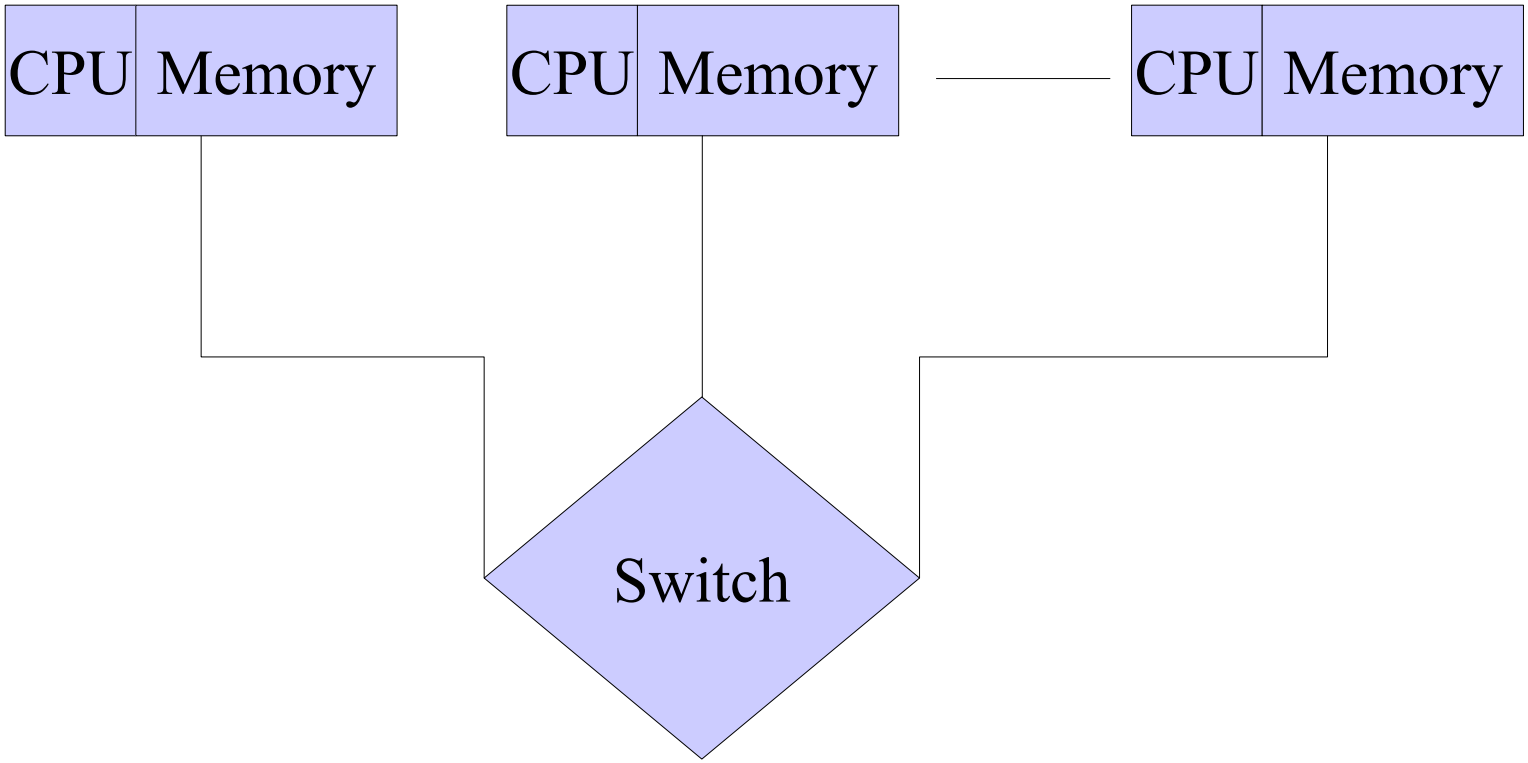
  –Shared memory (OpenMP)

  –Distributed memory (MPICH)

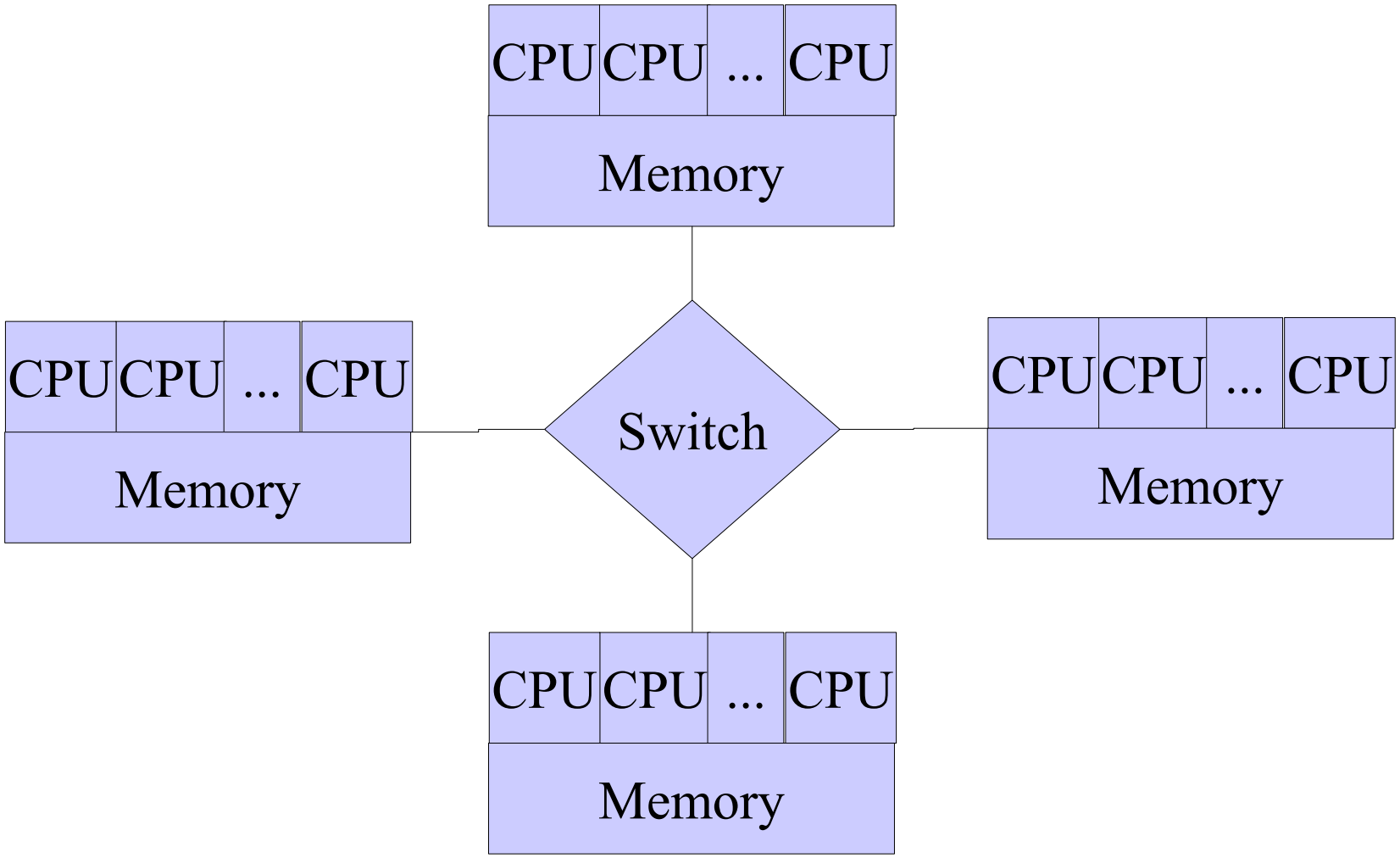
MIMD Shared Memory



MIMD Distributed



Why Hybrid Computing?



Computer Hardware Review (3)

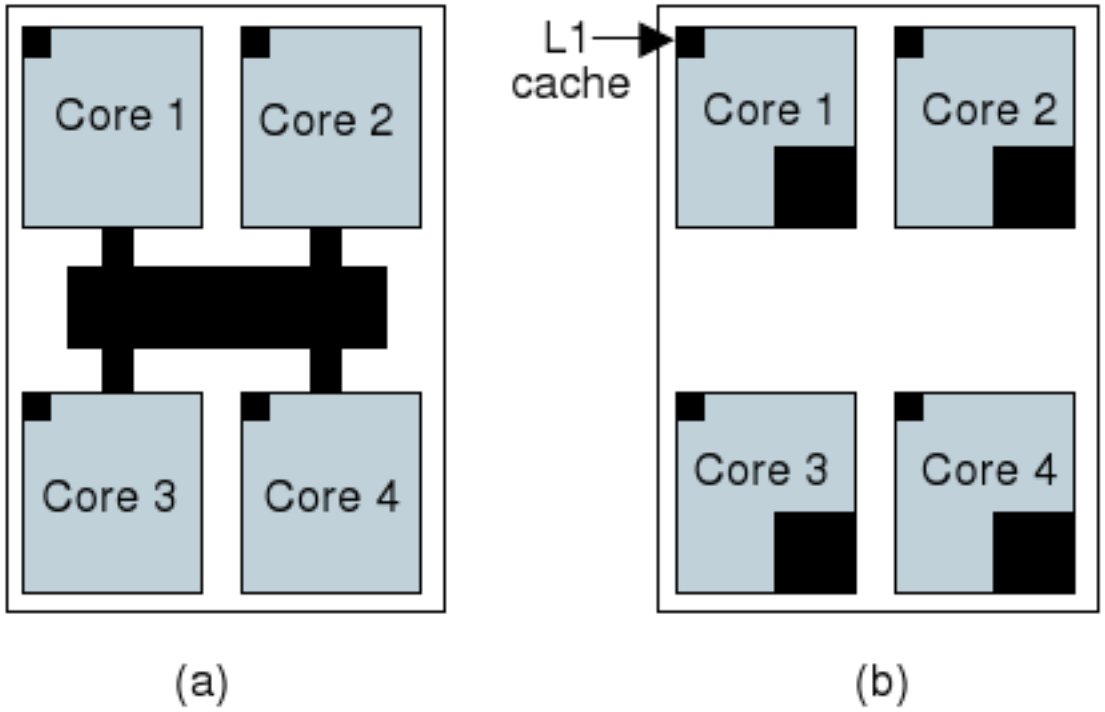
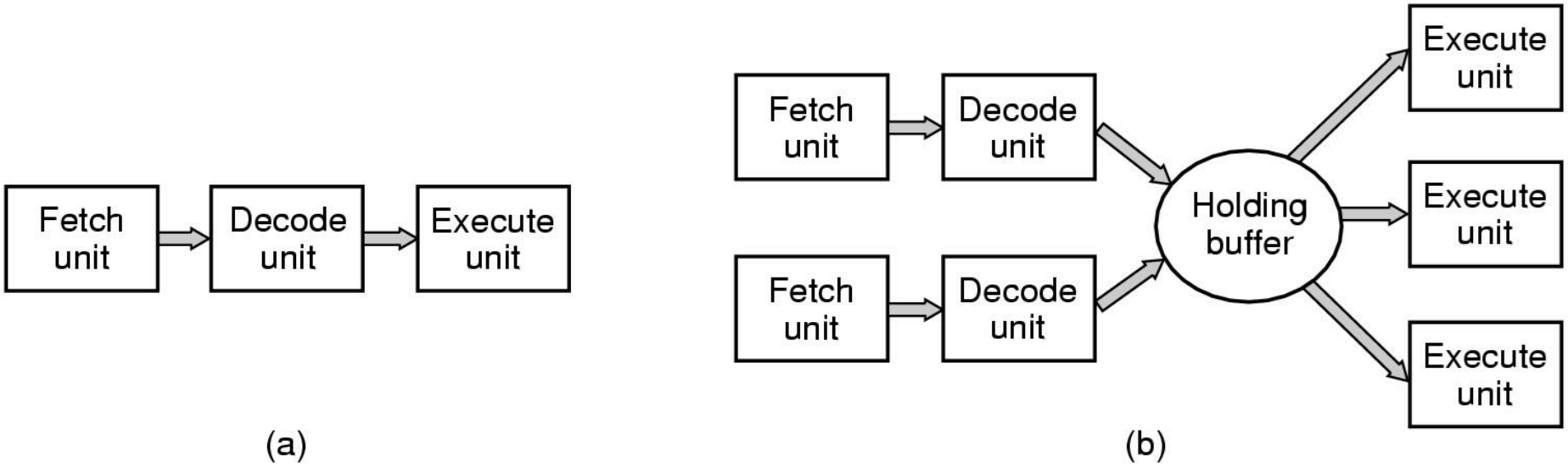


Figure 1-8. (a) A quad-core chip with a shared L2 cache. (b) A quad-core chip with separate L2 caches.

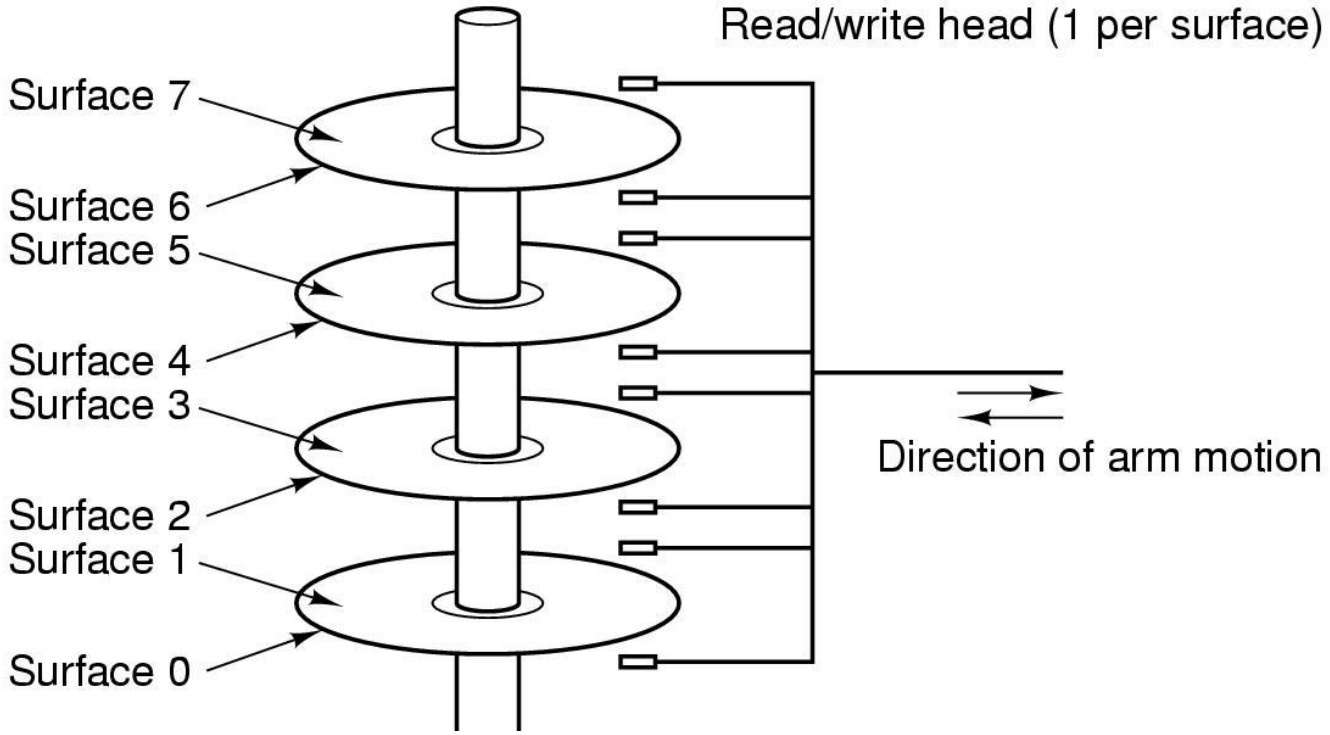
Computer Hardware Review (4)



(a) A three-stage pipeline

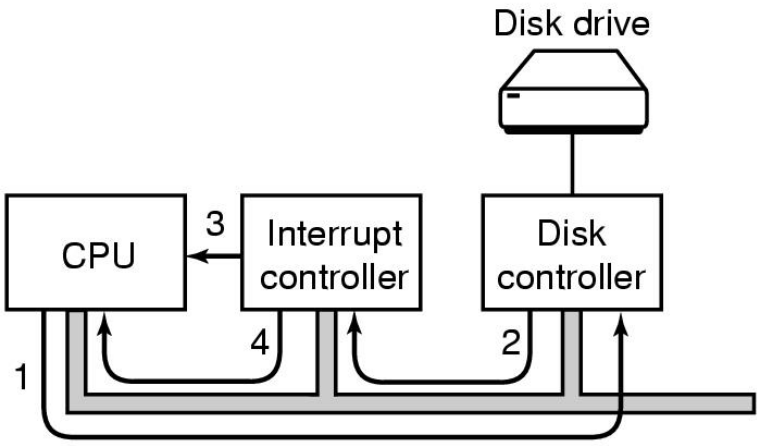
(b) A superscalar CPU

Computer Hardware Review (5)

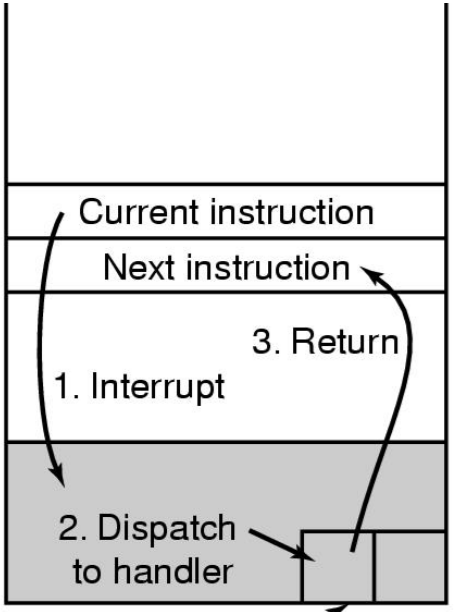


Structure of a disk drive

Computer Hardware Review (7)



(a)

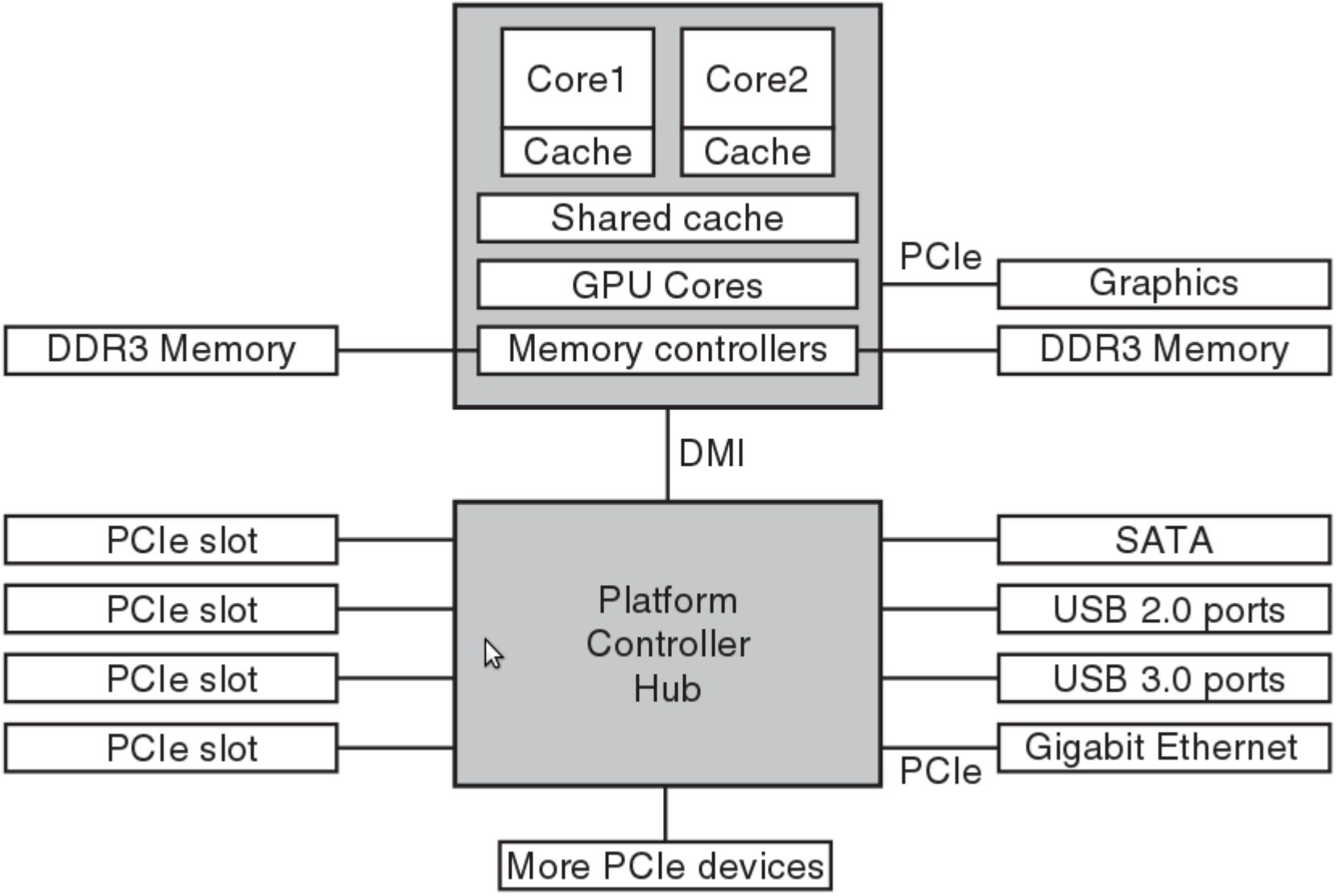


(b)

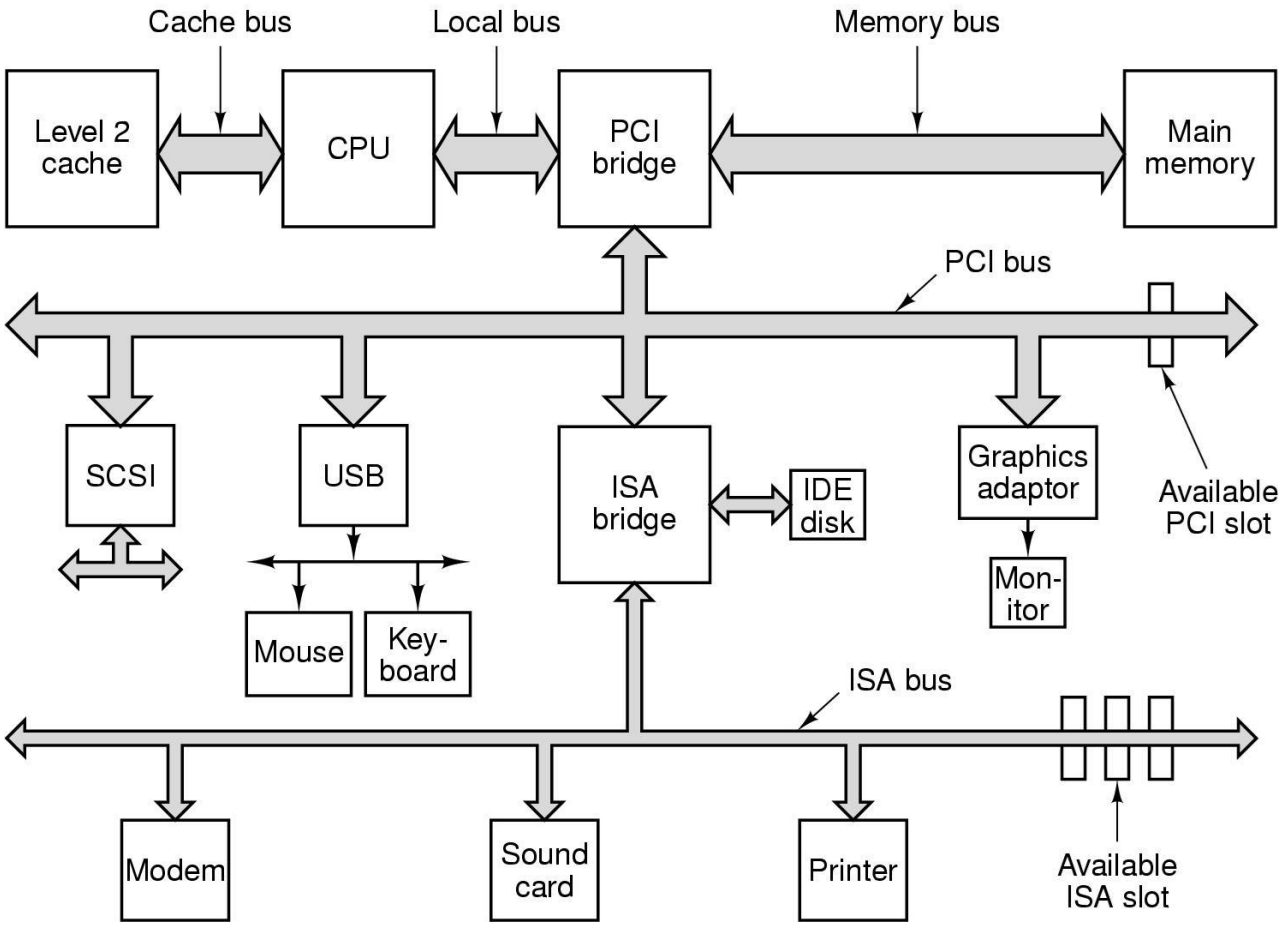
(a) Steps in starting an I/O device and getting interrupt

(b) How the CPU is interrupted

Computer Hardware Review (7)

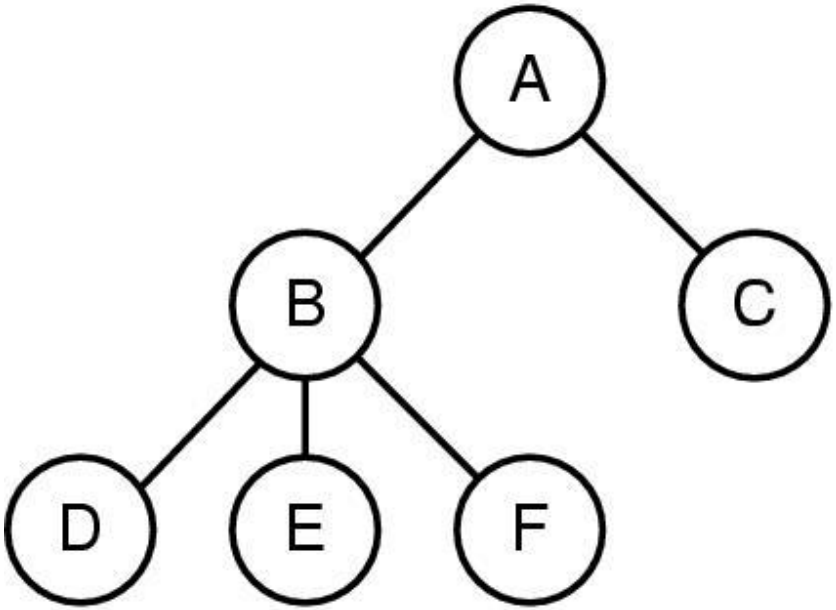


Computer Hardware Review (8)



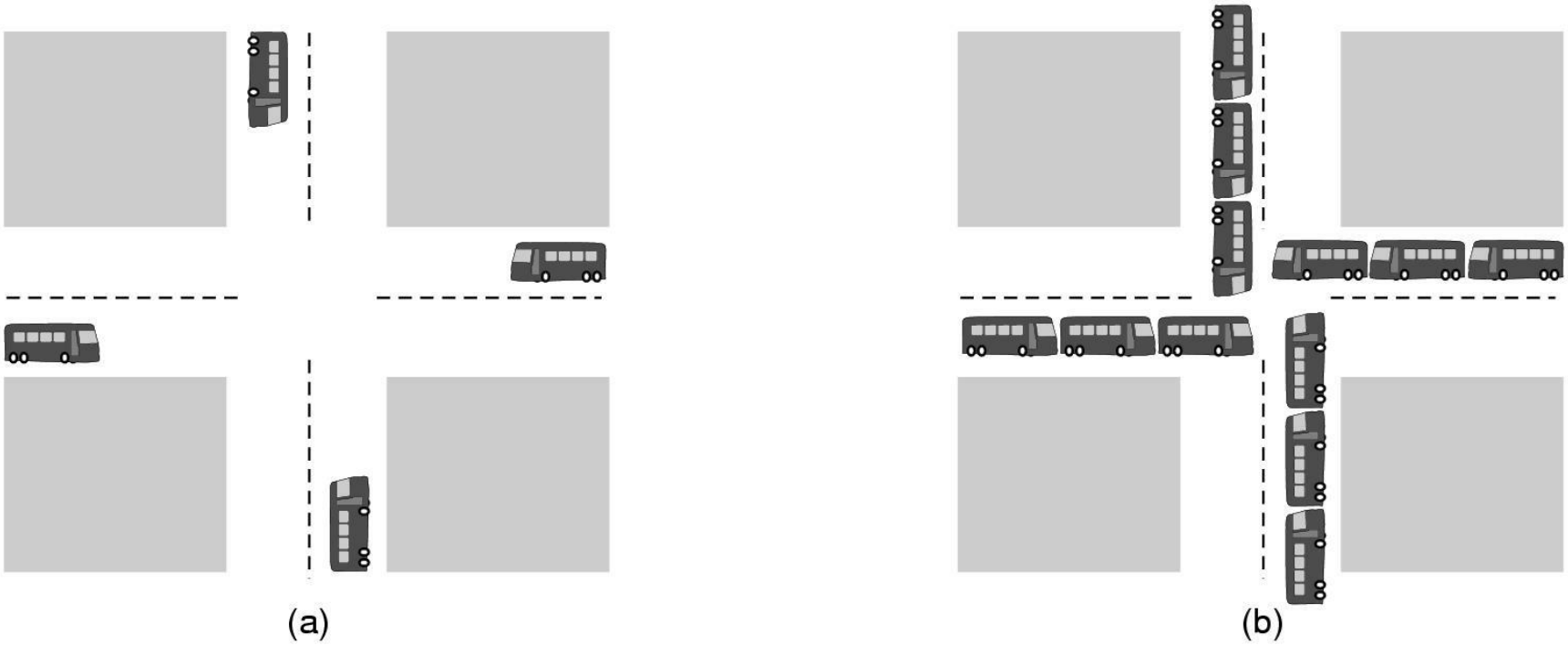
Structure of a large Pentium system

Operating System Concepts (1)



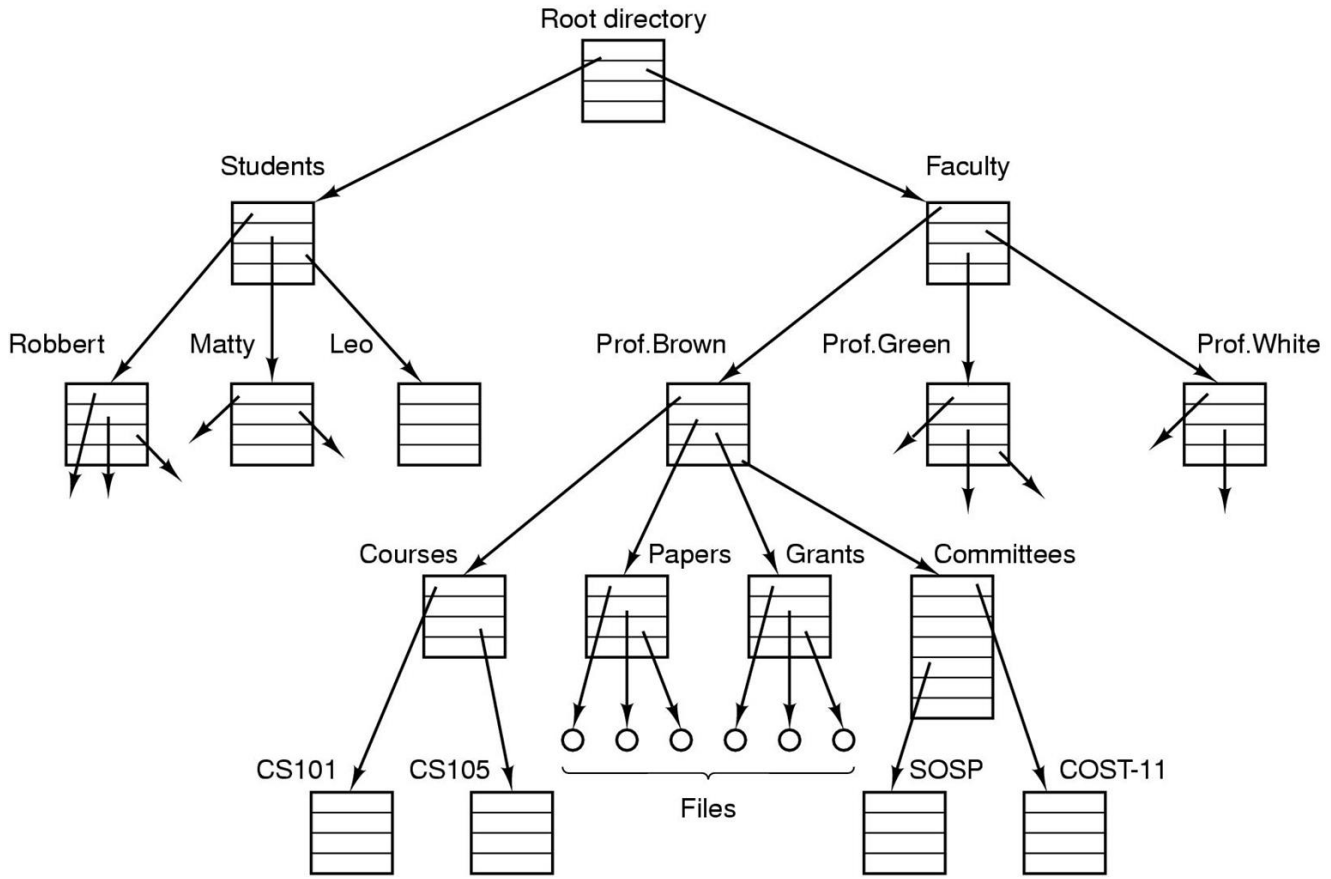
- A process tree
  - –A created two child processes, B and C
  - –B created three child processes, D, E, and F

Operating System Concepts (2)



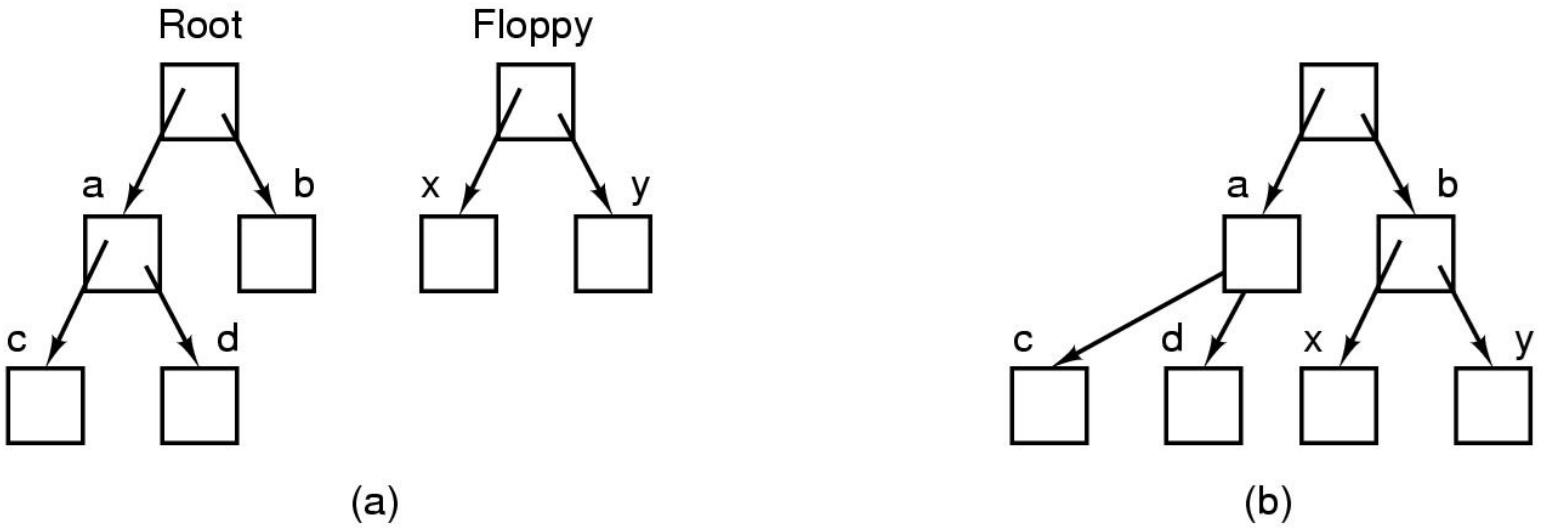
(a) A potential deadlock. (b) An actual deadlock.

Operating System Concepts (3)



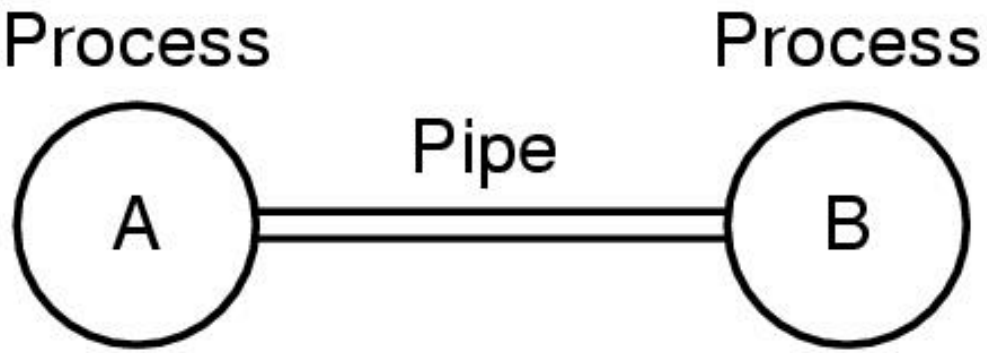
File system for a university department

Operating System Concepts (4)



- Before mounting,
  –files on floppy are inaccessible
- After mounting floppy on b,
  –files on floppy are part of file hierarchy

Operating System Concepts (5)

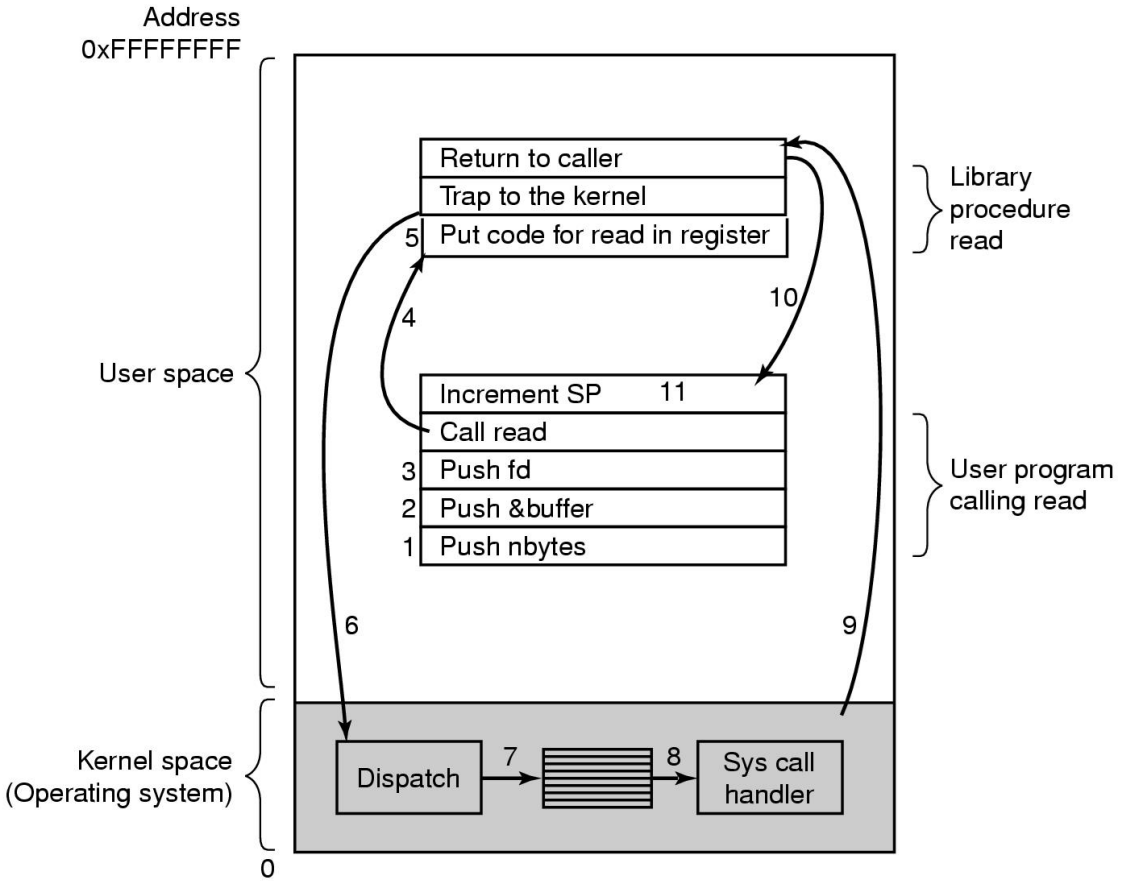


Two processes connected by a pipe

Examples

- date >file
- sort <file1 >file2
- cat file1 file2 file3 | sort >/dev/lp
- cat file1 file2 file3 | sort >/dev/lp &

Steps in Making a System Call



There are 11 steps in making the system call

count = read (fd, buffer, nbytes)

Some System Calls For Process Management

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Some System Calls For File Management

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Some System Calls For Directory Management

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Some System Calls For Miscellaneous Tasks

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

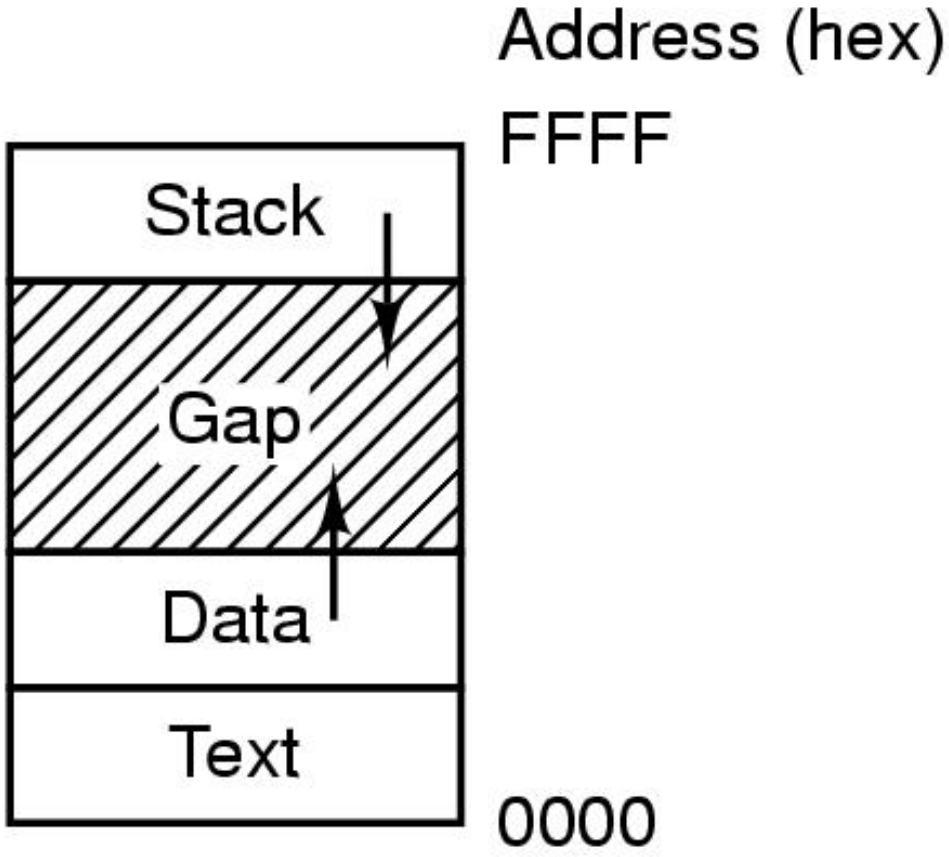
System Calls (1)

A stripped down shell:

```
#define TRUE 1
while (TRUE) {                                  /* repeat forever */
    type_prompt( );                             /* display prompt */
    read_command (command, parameters)          /* input from terminal */

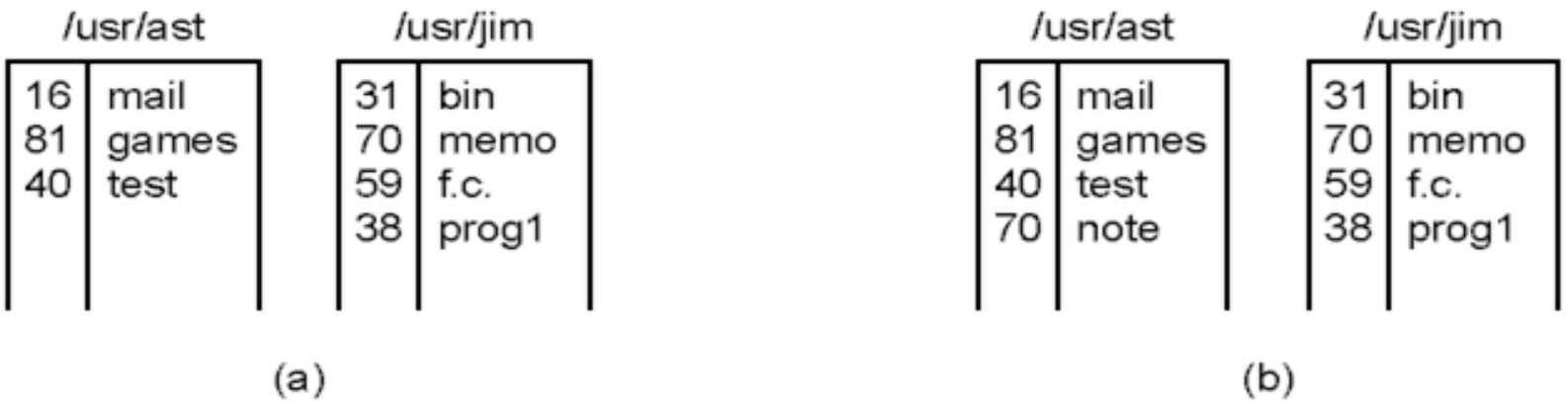
if (fork() != 0) {                              /* fork off child process */
    /* Parent code */
    waitpid( -1, &status, 0);                   /* wait for child to exit */
} else {
    /* Child code */
    execve (command, parameters, 0);            /* execute command */
 }
}
```

System Calls (2)



Processes have three segments: text, data, stack

System Calls (3)

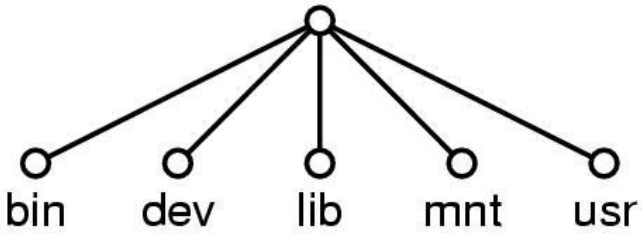


(a) Two directories before linking *usr/jim/memo* to ast's directory

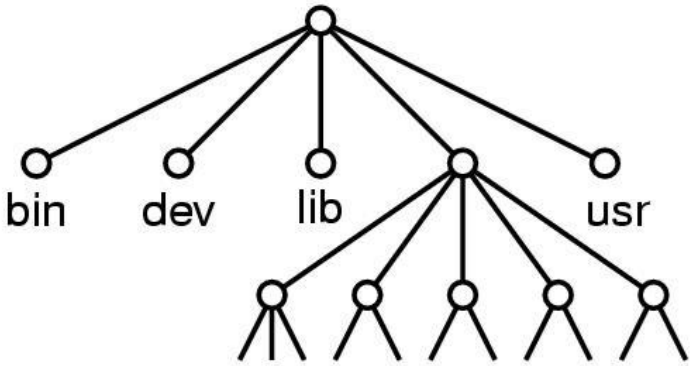
(b) The same directories after linking

```
link("/usr/jim/memo", "/usr/ast/note");
```

System Calls (4)



(a)



(b)

(a) File system before the mount

(b) File system after the mount

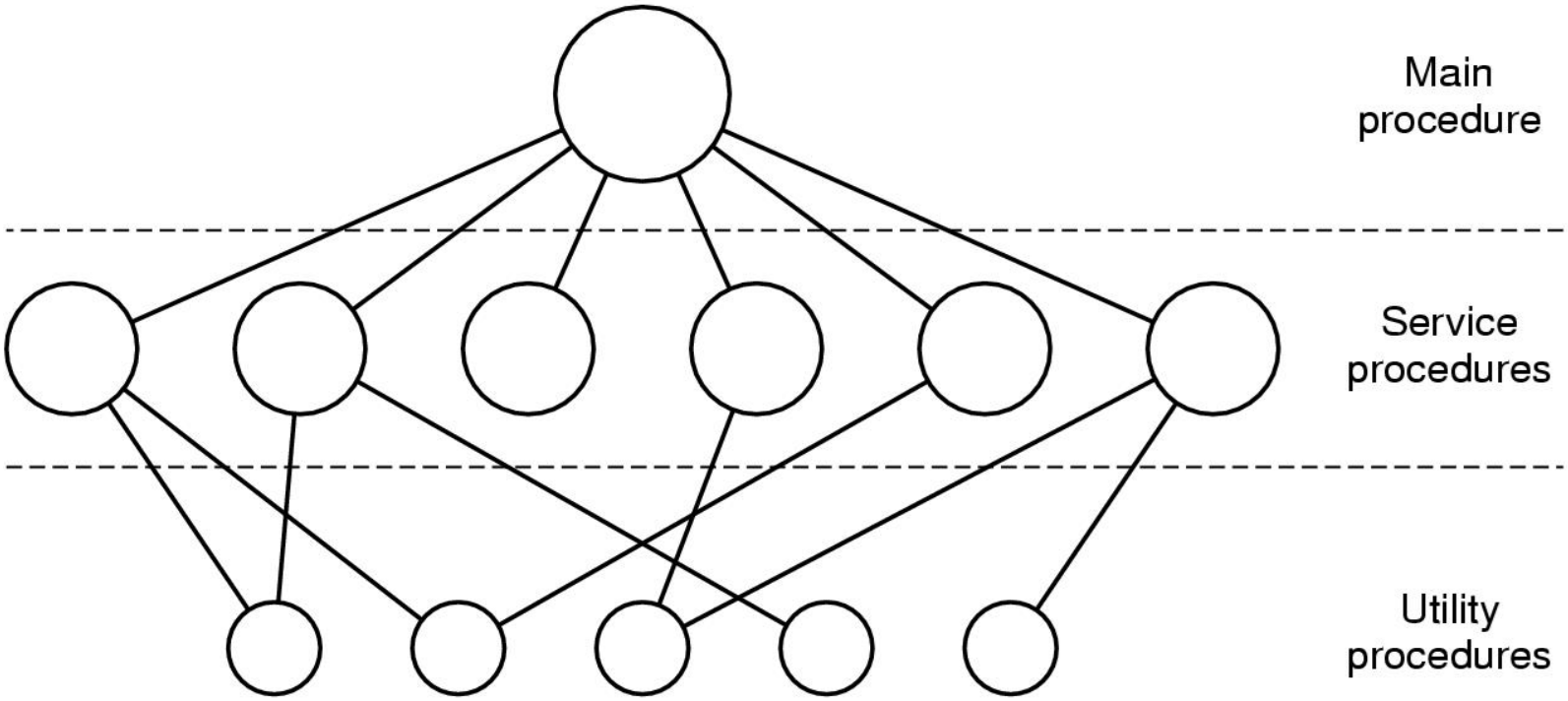
```
mount("/dev/sdb0", "/mnt", 0);
```

System Calls (5)

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Some Win32 API calls

Operating System Structure (1)



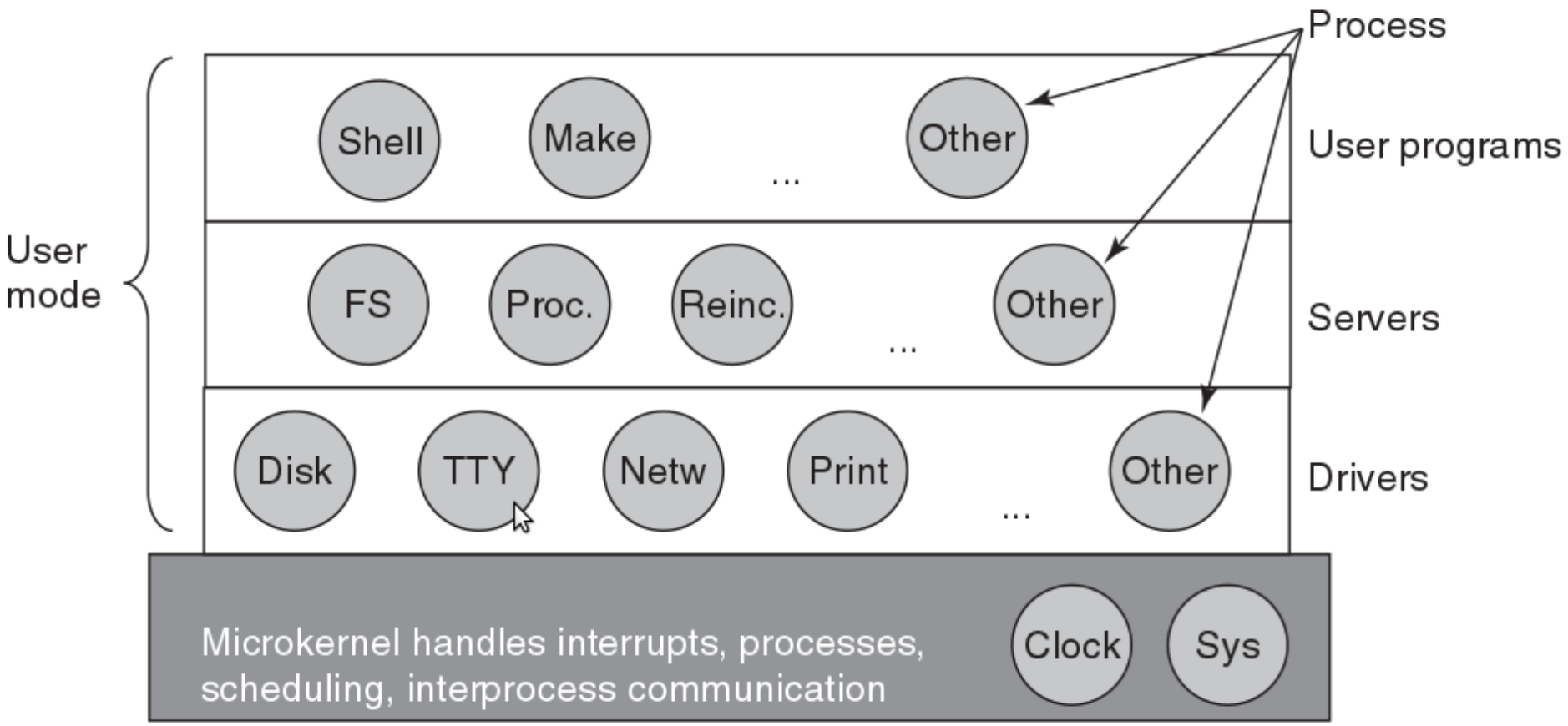
Simple structuring model for a monolithic system

Operating System Structure (2)

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

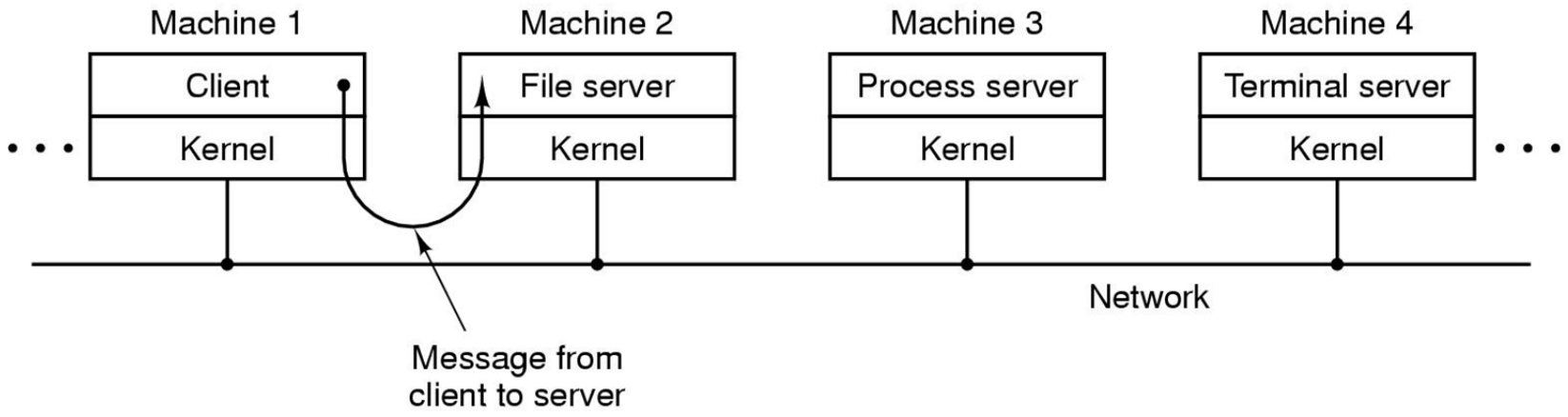
Structure of the THE operating system

Operating System Structure (3)



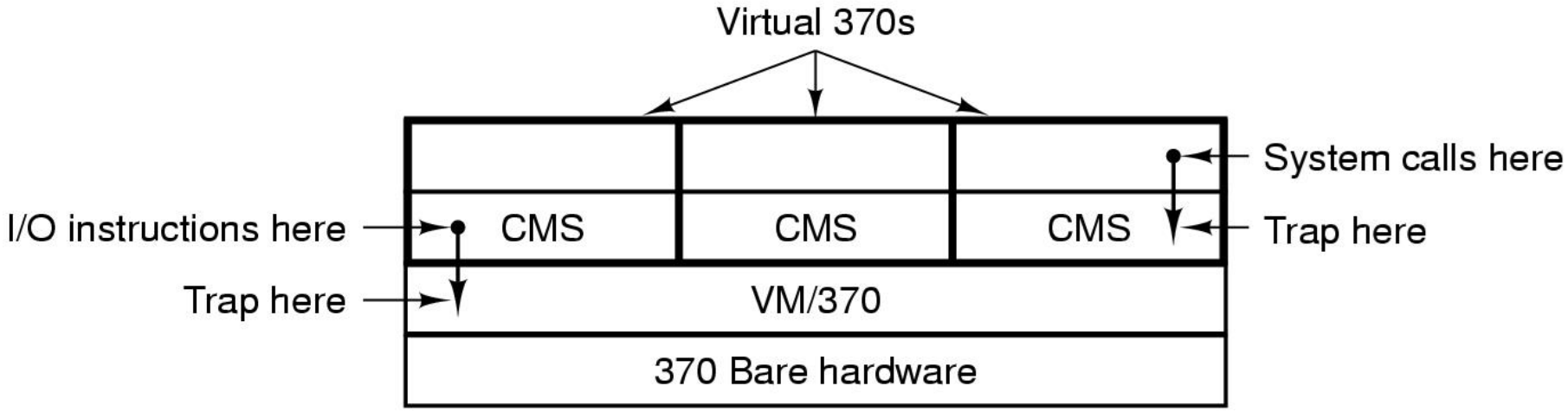
Simplified version of MIMIX system

Operating System Structure (4)



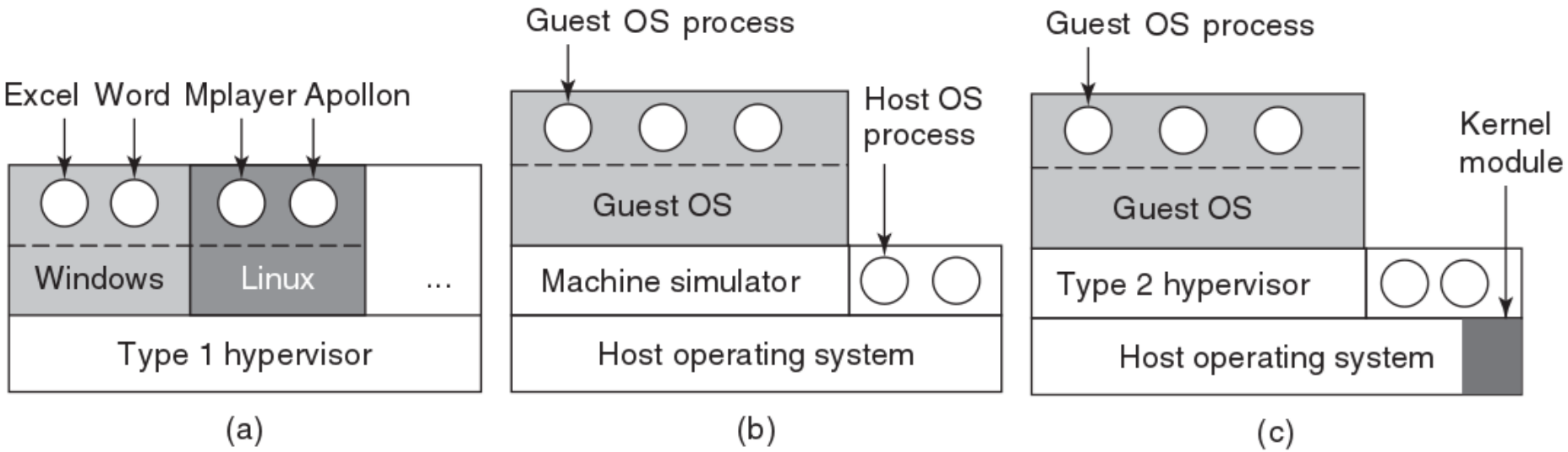
The client-server model over a network

Operating System Structure (5)



Structure of VM/370 with CMS

Hypervisors



(a) A type 1 hypervisor
(b) A pure type 2 hypervisor
(c) A practical type 2 hypervisor

Metric Units

Exp.	Explicit	Prefix	Exp.	Explicit	Prefix
10^{-3}	0.001	milli	10^{3}	1,000	Kilo
10^{-6}	0.000001	micro	10^{6}	1,000,000	Mega
10^{-9}	0.000000001	nano	10^{9}	1,000,000,000	Giga
10^{-12}	0.000000000001	pico	10^{12}	1,000,000,000,000	Tera
10^{-15}	0.000000000000001	femto	10^{15}	1,000,000,000,000,000	Peta
10^{-18}	0.000000000000000001	atto	10^{18}	1,000,000,000,000,000,000	Exa
10^{-21}	0.000000000000000000001	zepto	10^{21}	1,000,000,000,000,000,000,000	Zetta
10^{-24}	0.000000000000000000000001	yocto	10^{24}	1,000,000,000,000,000,000,000,000	Yotta

The metric prefixes